Assignment 3: The Quest for a Smarter Q-learning Agent

November 6, 2025

The Story Begins: The Birth of a Hero

Once upon a time in the vast world of Reinforcement Learning, agents were simple creatures. They wandered through small environments, learning by trial and error using a great spreadsheet known as the **Q-table**. For every possible situation (state) and action, they stored a number $Q(s, a)$—the expected total future reward.

They learned these numbers using the ancient Bellman equation:

$$Q(s, a) = \mathbb{E}\left[r + \gamma \max_{a'} Q(s', a')\right]$$

In simple terms, this meant: *"The value of doing a in s is the immediate reward r plus the best thing I can do next, discounted by γ."*

But these early agents faced a tragic limitation: their worlds had to be tiny. When they tried to explore larger realms—like Atari games—there were more possible states than stars in the galaxy. A table could no longer hold all that knowledge.

Then, a new hero emerged: the **Deep Q-Network (DQN)** [1]. Its insight was revolutionary:

> "What if we stop memorizing every Q-value and instead learn a function that *approximates* them?"

The Rise of the Deep Q-Network (DQN)

The DQN replaced the Q-table with a deep neural network $Q(s, a; \theta)$, parameterized by weights θ. This network took in a state s and predicted Q-values for all possible actions. It learned by minimizing the difference between its current prediction and a target value derived from the Bellman equation:

$$L(\theta) = \mathbb{E}\left[(Y_t - Q(s, a; \theta))^2\right]$$

where

$$Y_t = r + \gamma \max_{a'} Q(s', a'; \theta)$$

Through this process of prediction and correction, the DQN could learn directly from pixels and rewards—an achievement once thought impossible.

Soon, DQN agents were playing Atari games from raw screens and scoring higher than humans. A new era of deep reinforcement learning had begun.

The Hero's Problem: Chasing Its Own Tail

Yet even heroes have flaws. DQN's downfall was subtle but devastating—it learned from a moving target. In the target equation above, both the current estimate and the target value depend on the same network parameters θ. Every time the network updated its weights, the target also shifted.

This was like trying to hit a bullseye that moved whenever you threw the dart. The result? Instability, oscillation, and sometimes complete learning failure.

The Wise Twin: The Target Network

To restore balance, the DQN was given a **wise twin**—a second network that moved more slowly. We now had:

- **Policy Network:** $Q(s, a; \theta)$ — the fast learner, updated at every step.
- **Target Network:** $Q(s, a; \theta^-)$ — the calm twin, updated only occasionally.

Every C steps, the wise twin refreshed its knowledge: $\theta^- \leftarrow \theta$. This gave the main network a stable goal to aim for.

The updated target was now:

$$Y_t^{\text{DQN}} = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

This small but brilliant change turned chaos into order—and DQN could finally learn with stability.

The Hero's Secret Flaw: Optimism Bias

Even after learning stability, another flaw remained: DQN was a *chronic optimist*. The max operator in its target overestimated action values. Because neural networks make noisy predictions, the maximum often selected a lucky overestimate. This created an "optimism bias," leading the agent to chase phantom rewards.

The Dynamic Duo: Double DQN (DDQN)

The fix came from a wise mentor named **Double DQN (DDQN)** [2]. The key insight was to separate two decisions:

1. Which action looks best?
2. How valuable is that action really?

DQN used one network for both. DDQN used both twins cleverly:

- **Action Selection:** $a^* = \arg\max_{a'} Q(s', a'; \theta)$ (from the policy network)
- **Action Evaluation:** $Q(s', a^*; \theta^-)$ (from the target network)

The new target became:

$$Y_t^{\text{DDQN}} = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-)$$

By splitting the decision between two networks, DDQN reduced overestimation bias and learned more stable, grounded policies.

The Smart Memory: Prioritized Experience Replay (PER)

Even a wise duo can be inefficient if they forget what matters. DQN's memory—the **Replay Buffer**—stores past experiences $(s, a, r, s', \text{done})$ and replays them uniformly. This breaks correlations between samples, but it treats every experience as equally useful.

Think of how humans learn: we don't review everything equally—we revisit the problems we got wrong or found surprising. **Prioritized Experience Replay (PER)** [3] applies this same intuition: the agent should *learn more from mistakes*. It focuses training on transitions that were the most "surprising" or "confusing" in the past.

1. Measuring Surprise

Each experience's learning value is measured by its Temporal Difference (TD) error:

$$|\delta_i| = |Y_i - Q(s_i, a_i; \theta)|$$

Large errors indicate the agent's prediction was far from reality—these transitions deserve more attention.

2. Assigning Priorities

PER assigns every sample a priority based on this surprise:

$$p_i = (|\delta_i| + \epsilon)^\alpha$$

Here, ϵ ensures every sample has a nonzero chance of being chosen, and α controls how strongly we favor high-error samples:

- $\alpha = 0$: uniform sampling (no prioritization)
- larger α: stronger focus on surprising transitions

3. Correcting Sampling Bias

Since the buffer no longer samples uniformly, learning can become biased toward high-priority samples. To fix this, PER uses **importance sampling (IS) weights**. The sampling probability of each transition is:

$$P(i) = \frac{p_i}{\sum_k p_k}$$

and its IS weight is:

$$w_i = (N \cdot P(i))^{-\beta}$$

where N is the buffer size and β gradually increases toward 1 during training to fully correct the bias.

4. Learning and Updating

After each learning step, new TD errors are computed for the sampled transitions, and their priorities are updated. This means the memory itself evolves over time—what was once confusing may later become routine.

5. The Final Weighted Loss

The overall loss combines importance weights with the squared TD error:

$$L(\theta) = \mathbb{E}\left[w_i \cdot (Y_i - Q(s_i, a_i; \theta))^2\right]$$

This gives the agent an adaptive, self-tuning memory that focuses on the most informative experiences while keeping training unbiased.

In essence, PER transforms the replay buffer from a static archive into a living teacher—helping the agent spend more time learning from its own surprises.

Your Battlefield: The CartPole Environment

To test your agents, you will enter the world of **CartPole-v1**, a classic Gymnasium environment [4]. Official documentation: `https://gymnasium.farama.org/environments/classic_control/cart_pole/`

In this world:

- **State (s):** 4 continuous variables:
  1. Cart position
  2. Cart velocity
  3. Pole angle
  4. Pole angular velocity
- **Actions (a):**
  1. Push the cart left (0)
  2. Push the cart right (1)
- **Reward (r):** +1 for every step the pole remains upright.
- **Termination:** When the pole falls (beyond 12°) or the cart leaves the screen.
- **Maximum episode length:** 500 steps.

Goal: Keep the pole balanced for as long as possible, achieving the highest average reward.

Your Mission Tasks

You are provided with a nearly complete code base. Your mission is to bring these heroes—DQN, DDQN, and PER—to life by filling in the missing logic.

Tasks to Complete

1. **Target Q-Value Computation (`agent.py`)**
   - Implement DQN and DDQN target calculations.

2. **Bellman Target** (`agent.py`)
    - Combine r and the discounted Q-value to form the final target Y_t, handling terminal states.
3. **Importance Sampling Weights** (`memory.py`)
    - Compute $P(i)$ and w_i.
4. **Priority Update** (`memory.py`)
    - Update replay priorities using $p_i = (|\delta_i| + \epsilon)^\alpha$.
5. **TD Error Recalculation** (`agent.py`)
    - After each update, recompute $|\delta_i| = |Y_t - Q(s_i, a_i)|$ for replay priority updates.

Environment Setup

A helper script `setup_env.sh` is provided to create and configure the required conda environment. Run the script once before running any experiments:

```
bash setup_env.sh
```

This script will:

- Check for an existing conda installation. If conda is missing on Linux or macOS, it will download and install Miniconda automatically into $HOME/miniconda3 for the current user.
- Create a conda environment named `dqn_env` with Python 3.10.
- Install required packages: `numpy`, `torch`, `matplotlib`, `tqdm`, `swig`, and `gymnasium[box2d,other]`.

After running the script, activate the environment before running experiments:

```
conda activate dqn_env
```

Note for Windows users: Automated Miniconda installation is not included in this script for Windows. Please install Miniconda/Anaconda manually from `https://docs.conda.io/en/latest/miniconda.html`, then run the script from an Anaconda Prompt or Git Bash.

Running Experiments

Run all agents using: `python3 main.py --run_all`

This will sequentially train:

1. DQN
2. Double DQN (DDQN)
3. DQN + PER
4. Double DQN + PER

Videos will be saved in the `logs/` directory and training curves will be saved in the `results/` directory.

Deliverables

Submit a single PDF report containing:

- Implemented code snippets for all `TODO` sections.
- Training plots (`training_curves.png`, `performance_comparison.png`).
- **Analysis and Discussion:** Summarize your key learnings from implementing and testing DQN, DDQN, DQN+PER, and DDQN+PER on the CartPole environment. Discuss observed differences in their behavior, stability, and performance. You may also tune hyperparameters (e.g., learning rate, discount factor, replay buffer size, ϵ-greedy parameters) for deeper insights. *Note: Hyperparameter tuning is optional and not graded, but it can enhance your understanding.*

Important Instructions

- This is an **individual assignment**. Collaboration or code sharing is strictly prohibited.
- Deadline: **November 20, 2025, 11:59 PM**. Late submissions will not be accepted.
- Code and results must align; inconsistent submissions will not receive credit.

Good luck—and may your agents learn to balance both the pole and their optimism!

References

[1] Mnih, V., Kavukcuoglu, K., Silver, D., et al. *Human-level control through deep reinforcement learning.* Nature, 518(7540):529–533, 2015. arXiv:1312.5602

[2] Van Hasselt, H., Guez, A., and Silver, D. *Deep Reinforcement Learning with Double Q-learning.* Proceedings of AAAI, 2016. arXiv:1509.06461

[3] Schaul, T., Quan, J., Antonoglou, I., and Silver, D. *Prioritized Experience Replay.* ICLR, 2016. arXiv:1511.05952

[4] Towers, M., Terry, J. K., Balis, J. U., et al. *Gymnasium: A Standardized API for Reinforcement Learning Environments.* Farama Foundation, 2023. https://gymnasium.farama.org